

L10 Feed-forward 3D foundation models

Transformers from the ground up, then DUS3R in depth: one forward pass that returns what structure from motion estimated in six stages.

Feed-forward 3D foundation models

L10 · Unit 1

In L04 you built structure from motion one estimator at a time. Today a network does the whole job in one pass.

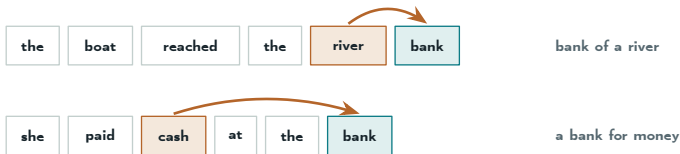
That network is a **transformer**. So we first build the transformer slowly, one idea at a time, and then take DUS3R apart carefully.

Today

1. **Transformers, from the ground up:** tokens, attention as a soft lookup, queries, keys and values, one example by hand, and the transformer block.
2. **Images as tokens:** the Vision Transformer, briefly.
3. **DUSt3R, in depth:** the pointmap, the network, the loss, and how to read cameras and matches out of its output.
4. **What was built on DUSt3R:** MAST3R, CUT3R, VGGT and the SLAM systems on top of them, one slide each.

Why a new kind of layer?

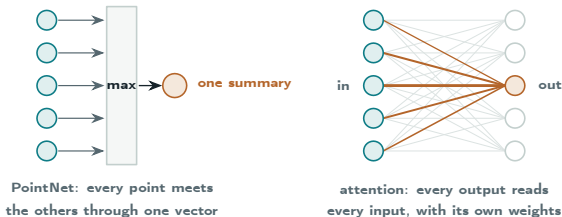
context changes meaning



- What *bank* means depends on the other words in the sentence.
- Pictures work the same way. Is a white patch a near wall or a far one? Is it the floor? You can only tell by looking at the rest of the image, and at the other image.
- We want a layer where **every piece of the input can look at every other piece** and update itself with what it finds. That layer is **attention**.

Tokens

a set of vectors



- A **token** is a vector of numbers standing for one piece of the input: a word, an image patch, a 3D point. Call its length d , the token's **width**. N tokens stacked as rows form an $N \times d$ matrix X .
- PointNet (L06) let points share information only through **one** pooled vector (left).
- Attention gives **every** token its own weighted reading of all the others (right).

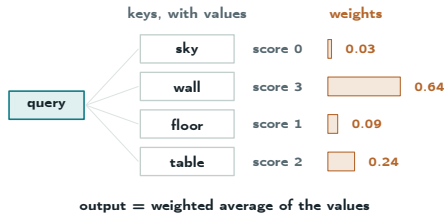
Attention is a soft dictionary lookup

A Python dictionary does a **hard** lookup:

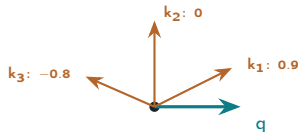
```
d = {"sky": v1, "wall": v2,
     "floor": v3, "table": v4}
d["wall"]    # exactly v2
```

The query must match one key exactly, and you get that key's value.

A **soft** lookup scores the query against *every* key, turns the scores into weights that sum to 1, and returns the **weighted average of all the values**. Here mostly the wall's value, some of the table's.



Scoring a match: the dot product



$$\text{score}(q, k) = q^T k = \|q\| \|k\| \cos \theta$$

- Pointing the same way: large and positive.
- At right angles: zero, unrelated.
- Pointing apart: negative.
- One multiply-add per number, so it is cheap to do for every pair of tokens.

Turning scores into weights: softmax

$$a_j = \frac{e^{s_j}}{\sum_k e^{s_k}}$$

- The exponential makes every score positive; dividing by the sum makes the weights add up to 1.
- The largest score gets the most weight, but not all of it. That is the *soft* in soft lookup.
- Scores $s = (2, 1, 0)$. What are the weights?

$$e^s = 7.39, 2.72, 1.00, \quad \text{sum} = 11.11$$

$$a = 0.67, 0.24, 0.09$$

Queries, keys and values

three roles for every token



$$q_i = W_Q x_i, \quad k_i = W_K x_i, \quad v_i = W_V x_i$$

- Each token makes a **query** (what am I looking for?), a **key** (what do I advertise to other queries?) and a **value** (what do I hand over if chosen?), each a vector of d numbers. The three matrices W are learned.
- To work out what *it* refers to, its query matches *animal*'s key best, so *animal*'s value gets most of the weight (0.50; weights illustrative).

Keys pick, values deliver

	key: what I advertise	value: what I hand over	weight
animal	a living noun	alive, singular, can tire	0.50
street	a place noun	a place, outdoors	0.18
tired	a state	tiredness	0.02
...	the other eight words		0.22

$$y_{it} = 0.50 v_{animal} + 0.18 v_{street} + 0.02 v_{tired} + \dots$$

- The **key** only *decides the weights*: the label on a folder. The **value** is *what gets copied*: its contents.
- So *it* comes out carrying mostly *animal's* value: now its vector says *alive, singular*.
- In DUS_t3R, a patch finds the same surface in image 2 by key; that patch's value hands over how camera 2 saw it.

Attention by hand

three tokens, $d = 2$

Queries and keys have $d = 2$ numbers each. $q_1 = (2, 1)$. Keys $k_1 = (1, 0)$, $k_2 = (0, 1)$, $k_3 = (1, 1)$. Values $v_1 = (1, 0)$, $v_2 = (0, 1)$, $v_3 = (1, 1)$.

■ Score, divide by $\sqrt{d}$, softmax, average the values

$$q_1^\top k_j = 2, 1, 3 \Rightarrow \div \sqrt{2}: 1.41, 0.71, 2.12$$

$$e^s: 4.11, 2.03, 8.34 \Rightarrow a_{1j} = 0.28, 0.14, 0.58$$

$$y_1 = 0.28 v_1 + 0.14 v_2 + 0.58 v_3 = (0.86, 0.72)$$

	k_1	k_2	k_3						
q_1	0.28	0.14	0.58	$\times$	1	0	$=$	0.86	0.72
q_2	0.11	0.45	0.45		0	1		0.55	0.89
q_3	0.58	0.14	0.28		1	1		0.86	0.42
	weights				values			outputs	

All the tokens at once

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad \text{Attention} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

```
def attention(X, Wq, Wk, Wv):  
    Q, K, V = X @ Wq, X @ Wk, X @ Wv  
    S = Q @ K.T / Q.shape[-1] ** 0.5  
    A = S.softmax(dim=-1)  
    return A @ V
```

X: N tokens, d numbers each
N x N scores
each row sums to 1
N weighted sums of values

- Row i of $QK^\top$ holds token i 's scores against every key: the slide before, for all N tokens in one matrix product.
- d is the length of each query and key: 2 in the example, 64 per head in real models. Dividing by $\sqrt{d}$ is explained next.
- That $N \times N$ matrix is the cost: double the tokens, four times the work.

Why divide by $\sqrt{d}$

■ How big is $q^\top k$ when each entry has mean 0 and variance 1?

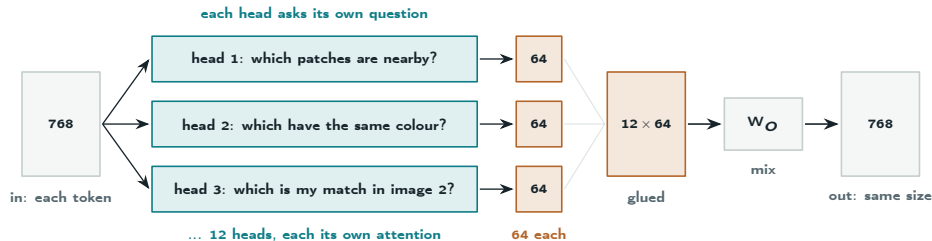
$$\text{Var}(q^\top k) = \sum_{i=1}^d \text{Var}(q_i k_i) = d$$

Typical size $\sqrt{d}$: 8 for $d = 64$. Dividing by $\sqrt{d}$ brings it back to about 1.

Without it, big scores make the softmax put nearly all the weight on one key (0.98, left), the lookup stops being soft, and the gradient vanishes: L05's saturated sigmoid again.



Many heads: several questions at once



- **In:** N tokens of 768 numbers. **Out:** N tokens of 768 numbers. Same shape, so blocks stack.
- **Inside:** 12 heads. Each has its own W_Q, W_K, W_V that shrink a token to 64 numbers, runs the attention you did by hand, and returns 64 numbers per token.
- Glue the answers back together, $12 \times 64 = 768$, and mix them with one more matrix W_O . Total cost: about one head of width 768.

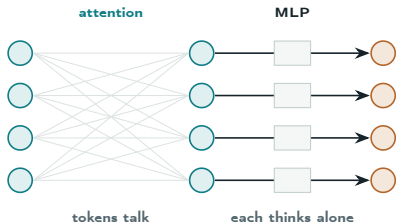
Attention ignores order

so we add position

- Shuffle the input tokens and the outputs come back shuffled the same way, otherwise unchanged. Nothing in the computation knows which token came first.
- For a point cloud that is exactly right (L06).
- For words or image patches it throws away where each piece was. So we **add a position encoding** to every token before the first layer, sines and cosines like NeRF's $\gamma(p)$ in L07, or a learned vector per position.

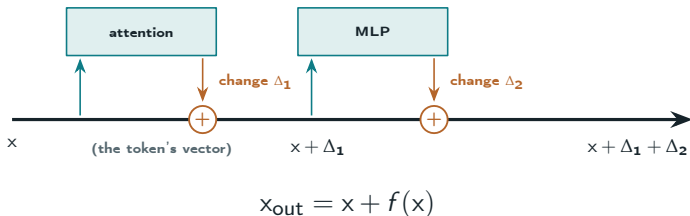


The transformer block: talk, then think



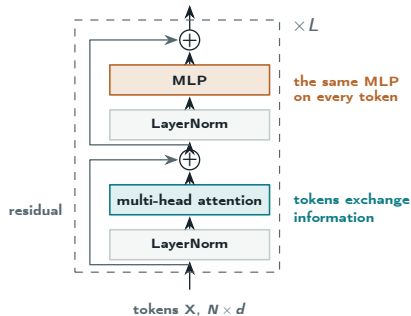
- **Attention:** the tokens talk. Each gathers what it needs from the others.
- **MLP:** each token thinks on its own about what it gathered, with the same small network for every token, like PointNet's shared MLP.
- A transformer repeats this block many times. Each round, every token's vector becomes a richer description of its piece in context.

Residual connections: edit, don't rewrite



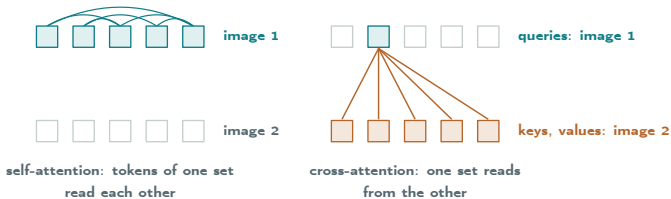
- Attention and the MLP do not replace a token's vector; each computes a **change** and adds it on. *bank* starts generic, and attention adds a nudge toward *river*.
- If a block has nothing useful to add, $f(x) \approx 0$ and the token passes through unchanged. Adding blocks cannot easily make things worse.
- For training: $\partial x_{\text{out}} / \partial x = I + \partial f / \partial x$. The I is a straight path back to the first layer, which L06's shrinking factors ($0.25^{10} \approx 10^{-6}$) cannot wipe out.

The block, precisely



- **Residual** connections, the two $\oplus$: each branch adds its change to the token (last slide).
- **Layer norm** (L06) keeps each token's numbers at a steady scale.
- In and out are both $N \times d$, so blocks stack: ViT-Large has 24.

Self-attention and cross-attention



- **Self-attention:** queries, keys and values all come from one set of tokens. The patches of one image talk among themselves.
- **Cross-attention:** queries from one set, keys and values from another. Each patch of image 1 asks image 2: *what do you have that relates to me?*
- Cross-attention is how DUS3R compares its two images.

Learn it again, differently

the explanations worth your time

- **3Blue1Brown**, Attention in transformers, step by step, with Transformers, the tech behind LLMs before it. The best pictures of vectors being refined by context.
 - **Jay Alammar**, The Illustrated Transformer. Queries, keys and values drawn one step at a time; the source of the *it* example.
 - **Andrej Karpathy**, Let's build GPT, from scratch, in code. Two hours; you will have written attention yourself by the end.
 - **Dive into Deep Learning**, Queries, keys and values. The soft-lookup view, with code.
- They explain language models, but the attention is identical. Swap *word* for *image patch*.

An image as tokens: the Vision Transformer

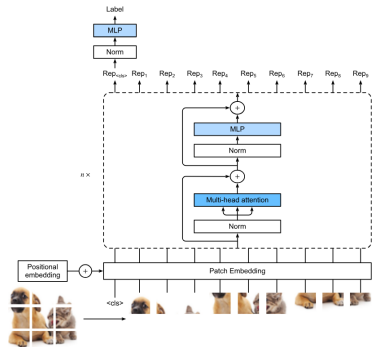
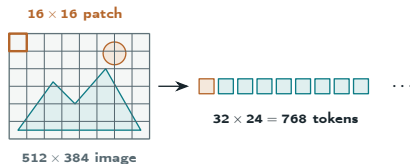


Figure: Dive into Deep Learning (d2l.ai), CC BY-SA 4.0

- Cut the image into 16×16 patches. Each patch is flattened and multiplied by a learned matrix: one token.
- Add a position encoding, then run the transformer blocks. That is all.



At DUS3R's 512×384 : $32 \times 24 = 768$ tokens, and $768^2 = 589,824$ attention weights per head per layer.

Learning to match without 3D labels: CroCo

Weinzaepfel et al., NeurIPS 2022

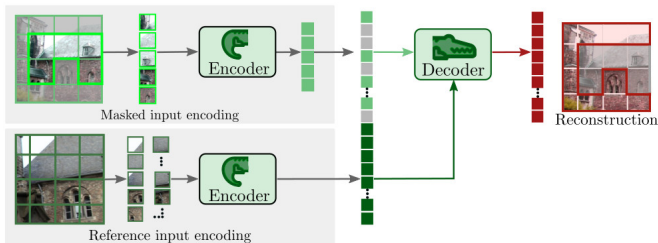


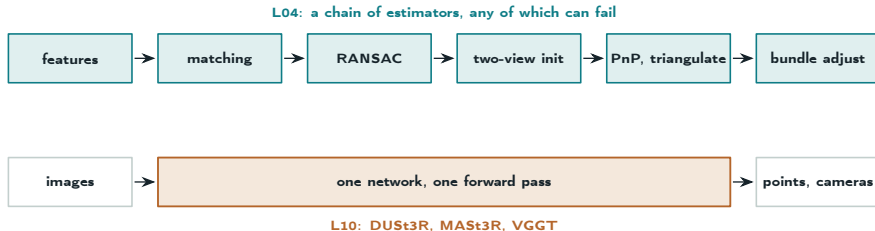
Figure: CroCo repository, Naver Corporation, CC BY-NC-SA 4.0

CroCo trains a ViT pair on one task: hide 90% of image 1's patches (top), show image 2 of the same scene (bottom), and paint the hidden patches back (right).

That needs finding the surface in image 2 and seeing it from viewpoint 1: correspondence and geometry, with no 3D labels. DUS_t3R, next, starts from these weights.

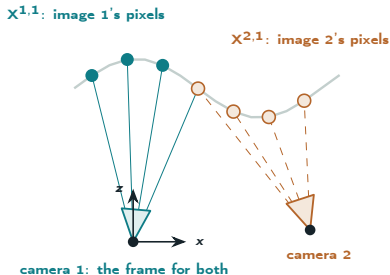
The problem DUS_t3R takes on

Wang et al., CVPR 2024



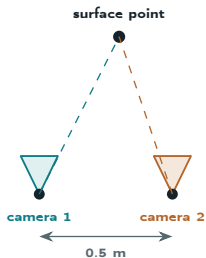
- **In:** two images. No intrinsics, no poses, no matches.
- **Out:** 3D geometry for every pixel of both, in one frame, plus a confidence for each.
- L04's chain breaks at its weakest stage: textureless walls, wide baselines, an unknown lens. DUS_t3R replaces the chain with one network that has seen millions of scenes.

The output: a pointmap



- A **pointmap** $X \in \mathbb{R}^{H \times W \times 3}$ stores one 3D point per pixel: where the surface seen at that pixel is.
- DUST3R predicts two: $X^{1,1}$, the pixels of image 1, and $X^{2,1}$, the pixels of image 2, **both in camera 1's frame**. Read the superscript as *image, frame*.
- Plus a **confidence** C at every pixel.

Why the second pointmap holds the pose



Camera 2 is 0.5 m right of camera 1, facing the same way. A point is at $(0.3, 0, 2)$ m in camera 1's frame.

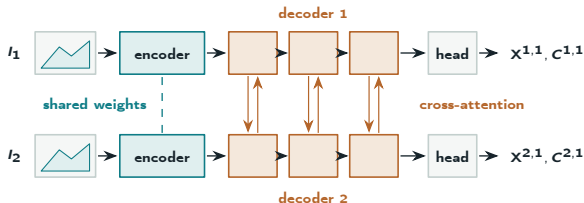
■ Where is it in camera 2's frame? Which number does DUS_t3R output for that pixel of image 2?

Camera 2's frame: $(0.3 - 0.5, 0, 2) = (-0.2, 0, 2)$, that is $X^{2,2}$. DUS_t3R must output $X^{2,1} = (0.3, 0, 2)$.

Image 2 alone gives the first. The second needs the 0.5 m offset.

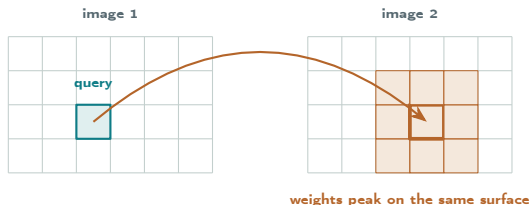
$X^{2,1} = R X^{2,2} + t$: the pose is baked into the second pointmap.

The network, one tensor at a time



Stage	Shape, per image	What happens
Patches Encoder, ViT-Large	$768 \text{ tokens} \times 1024$ 768×1024	512×384 image cut into 16×16 patches 24 blocks of self-attention; the same weights for both images
Two decoders, ViT-Base	768×768	12 blocks: self-attention, cross-attention to the other image , MLP
Head (DPT)	$384 \times 512 \times 4$	tokens back to pixels: X, Y, Z and confidence

Inside a decoder: cross-attention as soft matching



- A patch of image 1 sends its query to all 768 patches of image 2.
- After training, the weights peak on the patch showing **the same surface**: a soft match, with no descriptors and no RANSAC.
- What comes back tells the patch where the other camera saw that surface: what it needs to place its point in 3D.

Every decoder block does this, in both directions: $768 \times 768 = 589,824$ weights per head per block.

What a pointmap hides

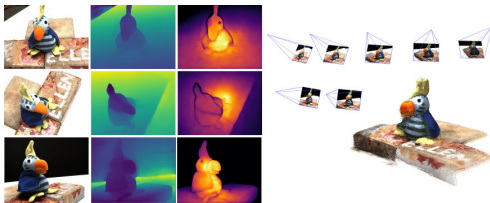
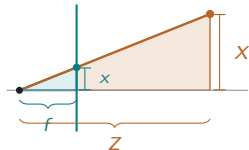


Figure: DUSt3R repository, Naver Corporation, CC BY-NC-SA 4.0

Quantity	How to read it off the pointmaps
Depth of image 1	the Z channel of $X^{1,1}$
Focal length	the f that best projects $X^{1,1}$ back onto its own pixels (next slide)
Relative pose	align the two frames' copies of the same pixels, as on the worked slide
Correspondences	pixels of the two images whose 3D points are mutual nearest neighbours

Focal length from a pointmap

Take square pixels, the principal point at the centre, and pixel coordinates (u, v) measured from it. A pinhole camera (L01) sends (X, Y, Z) to $f(\tilde{x}, \tilde{y})$, with $\tilde{x} = X/Z$ and $\tilde{y} = Y/Z$.



■ Least squares for f over all pixels, then two pixels worked

$$f^* = \arg \min_f \sum_i \|(u_i, v_i) - f(\tilde{x}_i, \tilde{y}_i)\|^2 \Rightarrow f^* = \frac{\sum_i u_i \tilde{x}_i + v_i \tilde{y}_i}{\sum_i \tilde{x}_i^2 + \tilde{y}_i^2}$$

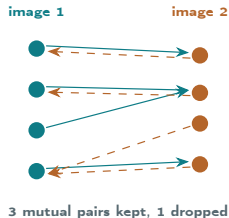
$(\tilde{x}, \tilde{y}) = (0.2, 0.1)$ at pixel $(101, 49)$, and $(-0.15, 0.1)$ at pixel $(-74, 51)$:
 $f^* = 41.3/0.0825 = 501$ pixels.

Pose and matches from pointmaps

- **Relative pose:** run the network on (l_1, l_2) and on (l_2, l_1) . Then $X^{1,1}$ and $X^{1,2}$ are the same pixels in two frames, and L07's Procrustes solve gives the pose, with a scale σ because pointmaps have none:

$$\min_{\sigma, R, t} \sum_i C_i^{1,1} C_i^{1,2} \|\sigma(RX_i^{1,1} + t) - X_i^{1,2}\|^2$$

- **Or PnP** (L04): the pixels of l_2 and their points $X^{2,1}$ in frame 1, with RANSAC.
- **Matches:** pixels whose points in $X^{1,1}$ and $X^{2,1}$ are **mutual** nearest neighbours in 3D.



The training loss

$$\ell_i = \left\| \frac{1}{z} \mathbf{x}_i - \frac{1}{\bar{z}} \bar{\mathbf{x}}_i \right\|$$

- Plain regression: the predicted point minus the true one, at every pixel of both pointmaps. $\bar{\mathbf{x}}$ comes from depth sensors and reconstructions, 8.5 million image pairs from eight datasets.
- z and $\bar{z}$ are the average distance of the predicted and true points to the origin. Dividing by them scales both to unit size, so the loss **ignores the overall size** of the scene.
- That is deliberate: two images alone cannot tell a room from a doll's house (L04). But it means DUS3R's output has **no metric scale**.

Letting the network say "I don't know"

$$\mathcal{L} = \sum_{v \in \{1,2\}} \sum_i C_i^v \ell_i^v - \alpha \log C_i^v, \quad C_i = 1 + e^{\tilde{C}_i} > 1$$

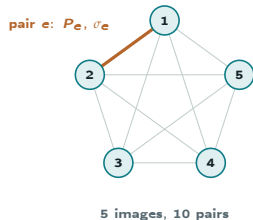
- The network predicts C itself. High C makes a pixel's error count more; the $-\log C$ term stops it from setting every C to its minimum.

■ What confidence does the loss reward at a pixel with error ℓ ?

$$\frac{\partial}{\partial C} (C\ell - \alpha \log C) = \ell - \frac{\alpha}{C} = 0 \quad \Rightarrow \quad C^* = \frac{\alpha}{\ell}$$

Small error, high confidence. Sky, glass, moving objects: C drops toward 1 and the pixel stops dominating the loss.

More than two images: global alignment



Run the network on pairs $e = (n, m)$. Find a pointmap χ^n per image in one world frame, and a pose P_e and scale σ_e per pair, that agree with every pair's output:

$$\min_{\chi, P, \sigma} \sum_e \sum_{v \in e} \sum_i C_i^{v,e} \|\chi_i^v - \sigma_e P_e \chi_i^{v,e}\|, \quad \prod_e \sigma_e = 1$$

■ Why the constraint on σ , and how many pairs for 20 images?

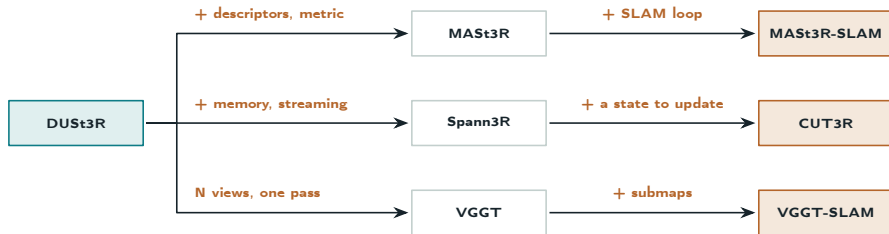
Without it, $\sigma_e = 0$ and $\chi = 0$ give zero cost.

$$\binom{20}{2} = \frac{20 \cdot 19}{2} = 190 \text{ pairs, each run in both orders}$$

What DUS3R leaves open

Limitation	Who fixed it
Matches are robust but a few pixels off	MASt3R: a descriptor head
No metric scale	MASt3R: a loss that keeps the scale
Pairs only; N^2 pairs, then an optimisation	VGGT: all views in one pass
Offline: needs all the images first	Spann3R, CUT3R: a memory, updated frame by frame
No SLAM: no keyframes, no loop closure	MASt3R-SLAM, VGGT-SLAM

Everything since builds on DUS3R



Same idea throughout: a transformer that outputs pointmaps. Each paper changes one thing.

MASt3R, then MASt3R-SLAM

Leroy et al., ECCV 2024; Murai et al., CVPR 2025

- **MASt3R** = DUS_t3R + a head that outputs a **descriptor** per pixel, trained so matching pixels have similar descriptors. Matches become pixel-accurate.
- It also keeps the scale in the loss when the training data is metric ($z := \bar{z}$), so its pointmaps are approximately **metric**.
- **MASt3R-SLAM** = L04's keyframe SLAM loop with MASt3R as the front end: track against the last keyframe, fuse, close loops. 15 frames per second, no calibration needed.



Figure: MASt3R repository, Naver Corporation, CC BY-NC-SA

4.0

Spann3R, then CUT3R: streaming

Wang and Agapito, 2024; Wang et al., CVPR 2025

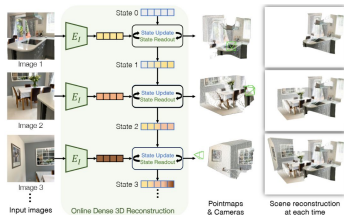
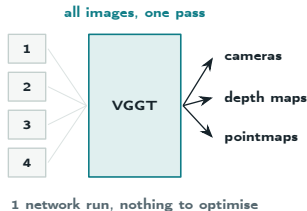
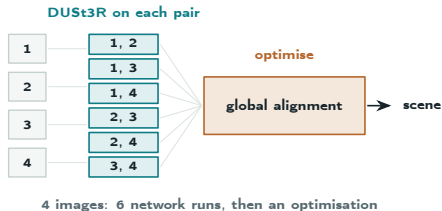


Figure: Wang et al., CUT3R (CVPR 2025), CC BY 4.0

- **Spann3R** = DUST3R + a **memory** of past frames, so each new frame's pointmap comes out directly in one global frame. No global alignment; it runs as images arrive.
- **CUT3R** replaces the memory with a fixed **state** of tokens that each frame reads and updates by cross-attention. Out come metric pointmaps and the pose.

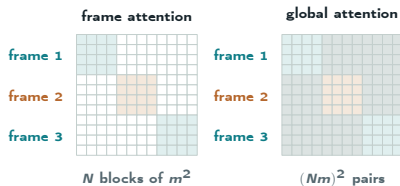
DUS3R vs VGGT: pairs, or all at once

Wang et al., CVPR 2025



	DUS3R	VGGT
Looks at	two images at a time	all N images together
Cameras	worked out afterwards, from the pointmaps	predicted directly, one per image
Many images	$\binom{N}{2}$ runs, then global alignment	one forward pass

How VGGT looks at all views at once



- Each image becomes patch tokens plus one **camera token**, and all of them go into one transformer.
- **Frame attention:** patches talk only within their own image (left), like DUS3R's self-attention.
- **Global attention:** every patch talks to every patch of every image (right). This replaces DUS3R's cross-attention between two images.
- VGGT alternates the two, 24 times each. A head reads each camera token out as a pose and focal length.
- The price: global attention is quadratic in *all* the tokens.

VGGT-SLAM

Maggio, Lim and Carlone, 2025

- VGGT cannot take a whole video at once: the quadratic cost again.
- **VGGT-SLAM** cuts the video into **submaps** of a few dozen frames, one VGGT pass each, and aligns the submaps in a factor graph (L04).
- Neighbouring submaps share a frame, which says how to line them up. The surprise is what kind of transformation that takes: next slide.

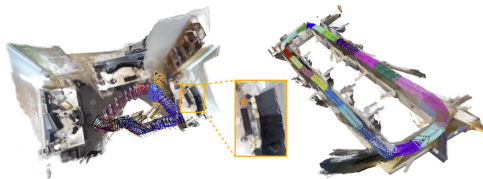
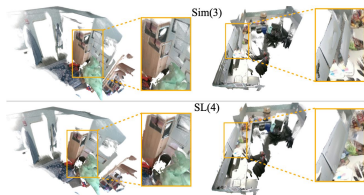


Figure: Maggio, Lim & Carlone, VGGT-SLAM (2025), CC BY 4.0

Lining up submaps: a similarity is not enough



true shape



Sim(3)
rotate, move, scale



projective, SL(4)
lines stay straight,
nothing else is kept

Figure: Maggio, Lim & Carlone, VGGT-SLAM (2025), CC BY 4.0

- With known cameras, two reconstructions of one place differ by a rotation, a translation and a scale: a **similarity**, $\text{Sim}(3)$, 7 numbers.
- With uncalibrated cameras each submap can also be **projectively warped**: straight lines stay straight, but angles, parallels and ratios do not (right). A similarity cannot undo that, and the walls come out doubled (left, top).

What $SL(4)$ is

A projective warp of 3D space is a 4×4 matrix acting on the point in homogeneous coordinates, then divided through (L02):

$$\begin{bmatrix} X' \\ Y' \\ Z' \\ w' \end{bmatrix} = H \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}, \quad \text{new point} = \left(\frac{X'}{w'}, \frac{Y'}{w'}, \frac{Z'}{w'} \right)$$

- H and $2H$ give the same point, so fix the scale with $\det H = 1$. The 4×4 matrices with determinant 1 are **$SL(4)$** , the *special linear group*: 16 entries minus 1 constraint, **15 numbers**, against $Sim(3)$'s 7.
- $SL(4)$ is a group, like $SO(3)$ and $SE(3)$, so L03's tools apply: VGGT-SLAM puts one H between neighbouring submaps in a factor graph, and closes loops the same way.

Three models side by side

	DUSt3R	MASt3R	VGGT
Input	an image pair	an image pair	1 to hundreds of images
Outputs	two pointmaps, confidence	pointmaps, confidence, descriptors	cameras, depth, pointmaps, tracks
Attention	cross-attention between two decoders	the same, plus a descriptor head	frame and global self-attention
Scale	none	approximately metric	none
Many views	global alignment over pairs	sparse global alignment on matches	one forward pass, BA optional

What happens to the SfM pipeline

L04 stage	In the feed-forward models	What survives
Features, matching RANSAC on E Two-view init, triangulation PnP registration	cross-attention; a descriptor head nothing inside the network pointmaps regressed directly	mutual nearest neighbours RANSAC around PnP and other fits nothing
Bundle adjustment	a camera head, or Procrustes on pointmaps global alignment, then nothing	PnP when a new image meets a map
Metric scale Calibration	a learned prior predicted fields of view	BA as refinement from the network's start a sensor: baseline, IMU, LiDAR, wheels known K still improves every result

Where they fail

- **Scale:** none unless trained for it, and learned scale is only as good as the training data's coverage of your camera and your scenes.
- **Memory:** attention is quadratic in tokens. More frames or more pixels run out of GPU memory quickly.
- **Out of distribution:** underwater, agricultural or surgical scenes, fisheye lenses, anything unlike the training sets.
- **Confidence is not uncertainty:** C ranks pixels; nothing makes it a calibrated probability. Unit 3 returns to this.
- **Moving objects** break global alignment, which assumes a static scene.

What you should be able to do

- Explain attention as a soft lookup, and compute one row of it by hand: dot products, softmax, weighted average.
- Say what queries, keys and values are for, why scores are divided by $\sqrt{d}$, and why a position encoding is needed.
- Count the tokens and attention weights for an image.
- Explain why DUS3R's second pointmap contains the relative pose, and how cross-attention lets it compute that.
- Recover focal length, pose and matches from DUS3R's pointmaps, and say what its loss does to scale and confidence.
- Place MAST3R, CUT3R, VGGT and their SLAM systems in the tree built on DUS3R.

Unit 1 is done

Unit 2 begins Monday

Unit 1 went from a catalog of representations to two ways of building one: per-scene optimisation (TSDF, NeRF, Gaussians) and feed-forward inference (today). Both give geometry without meaning.

- **Responses due Sun Oct 11:** Gaussian Splatting SLAM (L09) and DUST3R (today).
- **Next, L11 (Mon Oct 12):** lifting 2D foundation models into 3D. SAM 2, CLIP and DINOv2 features fused onto the maps built in this unit. Response paper: OpenScene.
- **Lab 1 is out today.** Project proposals, naming a target paper and an evaluation, are due **Fri Oct 23**.